

Fereastra cu actionare automata

1.Contributii:

Moise Cristian

- realizarea structurii mecanice a ferestrei automate;
- proiectarea si construirea cadrului din lemn si a sistemului de prindere pentru actuatorul liniar;
- participare la dezvoltarea software-ului pentru microcontroller-ul Arduino;

Ilinca Tudor-Ioan

- dezvoltarea interfetei grafice utilizand platforma Node-RED;
- implementarea comunicatiei prin protocolul MQTT intre dashboard si placa Arduino;
- realizarea conexiunilor si a montajului electric;

Macaneata-Vamos Andrei

- participare la dezvoltarea aplicatiei pentru microcontroller-ul Arduino;
- coordonarea etapelor de dezvoltare ale proiectului;
- realizarea arhitecturii software si proiectarea schemei electrice.

2.Objective:

Principalele obiective ale proiectului sunt:

- realizarea unui sistem automatizat pentru controlul unei ferestre;
- monitorizarea temperaturii în timp real;
- deschiderea automata a ferestrei atunci cand temperatura depaseste o valoare prestabila;
- inchiderea automata a ferestrei atunci cand temperatura scade sub pragul prestabilit;

- implementarea unui mod manual de control;
- transmiterea datelor prin protocol MQTT;
- afișarea parametrilor într-o interfață Node-RED Dashboard;
- dezvoltarea unui sistem simplu, modular și ușor de extins.

3.Descrierea domeniului ales și a soluțiilor similare

Domeniul ales pentru realizarea proiectului este reprezentat de aplicațiile Internet of Things (IoT) pentru automatizarea locuințelor. În cazul sistemelor de ventilație automată, monitorizarea temperaturii și controlul deschiderii ferestrelor contribuie la menținerea unui mediu confortabil și la reducerea consumului energetic.

În prezent, există numeroase companii care dezvoltă sisteme automate pentru controlul ferestrelor și al ventilației în cadrul locuințelor inteligente.

Compania Somfy dezvoltă sisteme smart home pentru automatizarea locuinței, oferind senzori, actuatori și control prin aplicații mobile pentru gestionarea ferestrelor, rulourilor și sistemelor de ventilație.

Compania VELUX oferă sisteme inteligente pentru controlul ferestrelor electrice de mansardă, utilizând senzori de temperatură, umiditate și CO₂ pentru automatizarea ventilației și monitorizarea climatului interior.

Proiectul nostru își propune să ofere o alternativă mai accesibilă din punct de vedere financiar, utilizând componente open-source și tehnologii ușor de implementat. Totodată, sistemul este ușor de scalat prin adăugarea unor senzori suplimentari pentru implementarea unor funcționalități avansate. De asemenea, datorită utilizării protocolului MQTT, sistemul poate fi integrat cu ușurință într-un ecosistem complet de tip smart home, permițând comunicarea cu mai multe dispozitive inteligente.

4.Descrierea solutiei propuse cu prezentarea functionalitatilor aferente solutiei

4.1 Arhitectura sistemului

Sistemul este format din urmatoarele componente principale:

- placa de dezvoltare Arduino UNO R4 WIFI;
- senzor de temperatura si umiditate DHT11;
- actuator liniar pentru deschiderea ferestrei;
- driver BTS7960 pentru controlul actuatorului;
- broker MQTT (laptop);
- platforma Node-RED pentru monitorizare si control.

Fluxul de functionare al sistemului in modul auto este urmatorul:

1. Senzorul DHT11 citeste temperatura mediului.
2. Arduino proceseaza valorile citite.
3. Daca temperatura depaseste pragul stabilit, actuatorul deschide fereastra.
4. Daca temperatura scade sub pragul stabilit, fereastra se închide.
5. Datele sunt transmise prin MQTT către Node-RED.

Utilizatorul poate controla sistemul manual din dashboard, caz in care temperatura este doar afisata si nu mai influenteaza actionarea ferestrei.

4.2 Functionalitatile sistemului

Sistemul implementat ofera urmatoarele functionalitati:

Monitorizarea temperaturii

Senzorul DHT11 masoara temperatura mediului si transmite periodic valorile catre microcontroler. Acesta le trimite broker-ului prin protocolul MQTT pentru vizualizarea dashboard-ului inteligent de pe orice dispozitiv ce este conectat la retea.

Mod automat

În modul automat, sistemul controlează fereastra în funcție de temperatura detectată.

Mod manual

Utilizatorul poate deschide sau închide manual fereastra din interfața Node-RED.

Comunicatie MQTT

Datele sunt transmise prin protocol MQTT, ceea ce permite integrarea sistemului în aplicații IoT.

Interfața grafică

Node-RED Dashboard afișează:

- temperatura curentă;
- starea ferestrei;
- modul de funcționare;
- comenzile manuale.

6. Testarea soluției

Testarea sistemului a fost realizată în mai multe etape pentru verificarea funcționării corecte a tuturor componentelor.

6.1 Testarea senzorului

S-a verificat citirea corectă a temperaturii și transmiterea valorilor către sistem.

6.2 Testarea actuatorului

S-a verificat funcționarea actuatorului liniar atât pentru deschiderea, cât și pentru închiderea ferestrei.

6.3 Testarea modului automat

S-a verificat reacția sistemului la modificarea temperaturii:

- la depășirea pragului stabilit, fereastra se deschide;
- la scăderea temperaturii sub prag, fereastra se închide.

6.4 Testarea comunicației MQTT

S-a verificat transmiterea corectă a mesajelor între Arduino și Node-RED.

6.5 Testarea interfeței grafice

S-a verificat afișarea corectă a datelor și funcționarea comenzilor manuale.

Rezultatele testelor au demonstrat funcționarea stabilă și corectă a sistemului.

7. Codul sursă Arduino IDE

```
#include <DHT.h>
#include <WiFiS3.h>
#include <PubSubClient.h>

#define DHTPIN 2
#define DHTTYPE DHT11

// WIFI
const char* ssid = "TP-Link_6AA8";
const char* password = "56943849";

// MQTT
const char* mqtt_server = "192.168.1.223";
const int mqtt_port = 1883;

// declarare client mqtt
WiFiClient wifiClient;
PubSubClient client(wifiClient);

DHT dht(DHTPIN, DHTTYPE);

const int RPWM = 5;
const int LPWM = 6;
bool open = true;
bool autoMode = true;

unsigned long lastMsg = 0;

// ===== Functii ===== //

void stopActuator() {
    analogWrite(RPWM, 0);
    analogWrite(LPWM, 0);
}

void openActuator() {
    analogWrite(RPWM, 200);
    analogWrite(LPWM, 0);
}
```

```

void closeActuator() {
    analogWrite(RPWM, 0);
    analogWrite(LPWM, 200);
}

// functie callback pentru mod AUTO/MANUAL
void callback(char* topic, byte* payload, unsigned int length)
{
    String msg = "";

    for (unsigned int i = 0; i < length; i++)
    {
        msg += (char)payload[i];
    }

    Serial.print("MQTT: ");
    Serial.println(msg);

    if (msg == "AUTO")
    {
        autoMode = true;
        Serial.println("Mod AUTO");
    }
    else if (msg == "MANUAL")
    {
        autoMode = false;
        Serial.println("Mod MANUAL");
    }
    else if (autoMode == false && msg == "OPEN")
    {
        openActuator();
        delay(16000);
        stopActuator();
        open = true;
    }
    else if (autoMode == false && msg == "CLOSE")
    {
        closeActuator();
        delay(16000);
        stopActuator();
        open = false;
    }
}

void connectWiFi()
{
    while (WiFi.begin(ssid, password) != WL_CONNECTED)
    {
        Serial.print(".");
        delay(2000);
    }
    Serial.println("\nWiFi conectat");
}

void reconnectMQTT()
{
    while (!client.connected())
    {

```

```

        Serial.println("Incerca conectare MQTT...");
        if (client.connect("arduinoClient"))
        {
            Serial.println("MQTT conectat");
            client.subscribe("fereastră/actuator/cmd");
        }
        else
        {
            Serial.print("Eroare MQTT, rc = ");
            Serial.println(client.state());
            delay(2000);
        }
    }
}

// ===== //

void setup()
{
    pinMode(RPWM, OUTPUT);
    pinMode(LPWM, OUTPUT);
    dht.begin();

    stopActuator();
    Serial.begin(115200);
    delay(300);

    connectWiFi();

    client.setServer(mqtt_server, mqtt_port);
    client.setCallback(callback);
}

void loop()
{
    if (!client.connected())
    {
        reconnectMQTT();
    }
    client.loop();

    float t = dht.readTemperature();

    if (isnan(t))
    {
        Serial.println("Eroare la citirea temperaturii");
        delay(500);
        return;
    }

    Serial.print("Temperatura: ");
    Serial.println(t);

    // trimitere date prin mqtt
    if (millis() - lastMsg > 2000)
    {
        lastMsg = millis();
    }
}

```

```

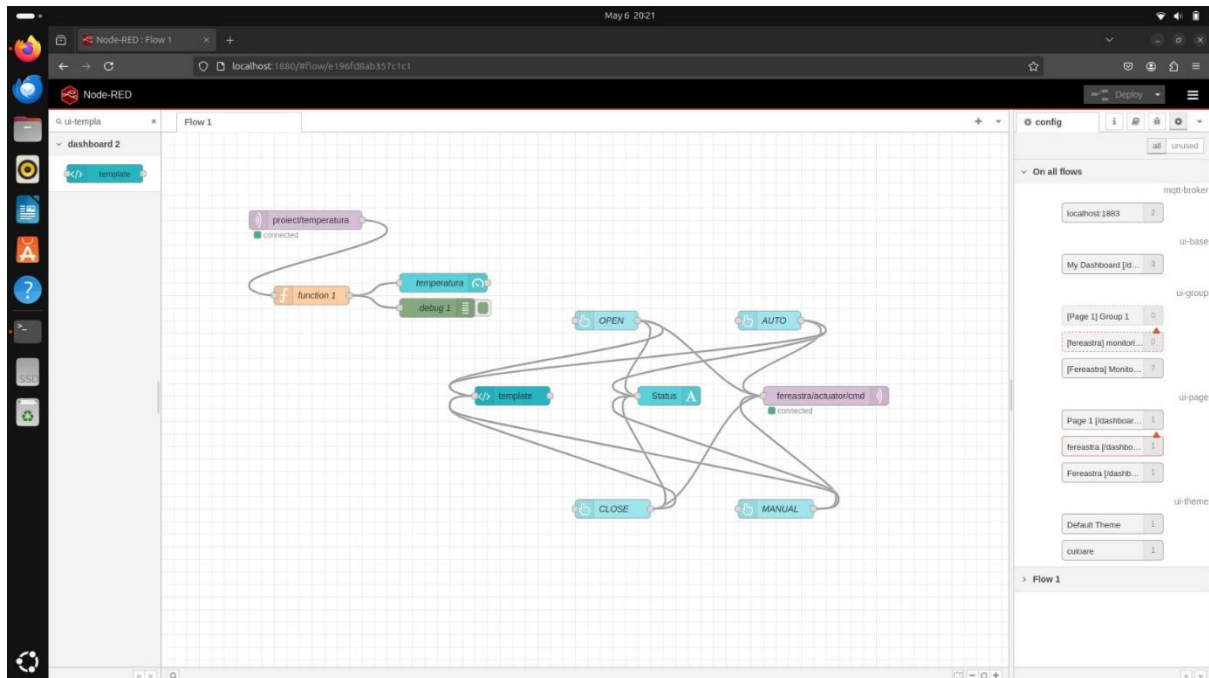
    char tempString[8];
    dtostrf(t, 1, 2, tempString);
    client.publish("proiect/temperatura", tempString);
}

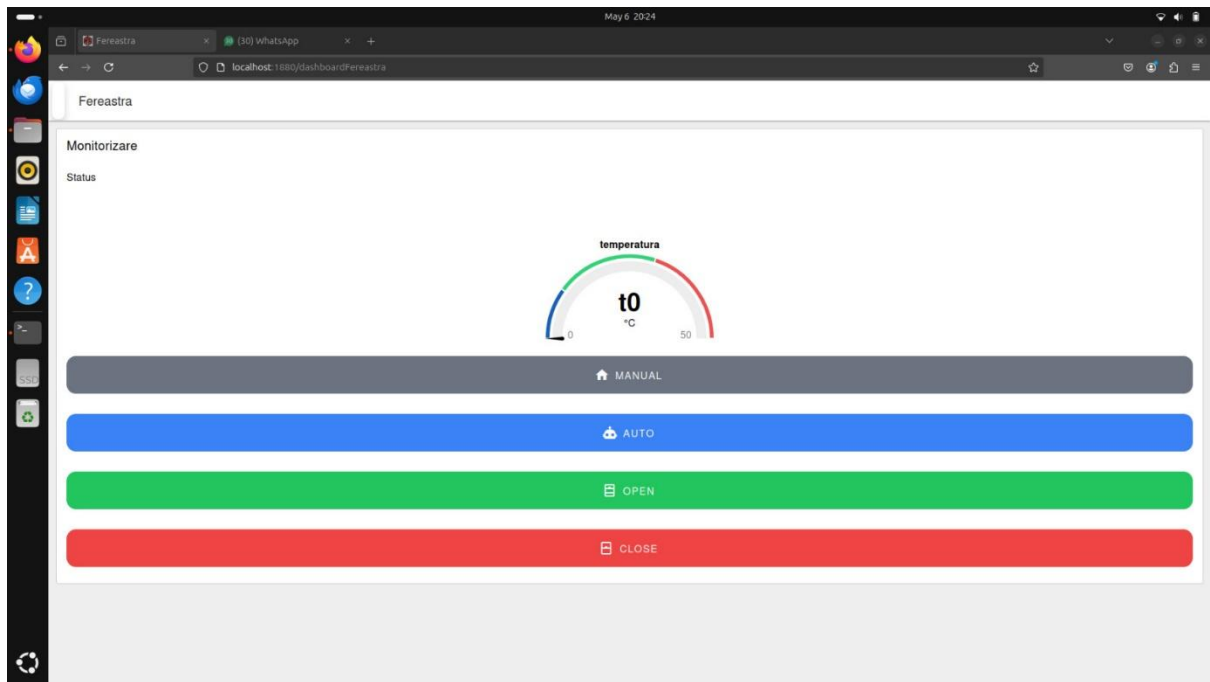
// logica pentru modul AUTO
if (autoMode == true)
{
    if (t > 30 && open == false)
    {
        openActuator();
        delay(13000);
        stopActuator();
        open = true;
    }
    else if (t < 28 && open == true)
    {
        closeActuator();
        delay(16000);
        stopActuator();
        open = false;
    }
}

delay(500);
}

```

8. Interfata Node-RED





9. Schema Electrica

